ENTERPRISE SYSTEM DOCUMENTATION

D R AGENCIES

GST Billing, Real-Time Inventory & Customer Ledger Management Portal

Comprehensive Technical & Functional Architecture Manual detailing all application modules, authentication workflows, inventory stock algorithms, Bluetooth thermal printing, and backend microservices.

APPLICATION VERSION
v1.0.0 (Production Stable)

MOBILE FRAMEWORK
React Native 0.79 / Expo SDK 53

BACKEND PLATFORM
Node.js Express / Supabase PostgreSQL

HARDWARE INTEGRATION
Bluetooth ESC/POS Thermal (58mm/80mm) & A4 Print

RELEASE DATE
September 2026

TARGET ENVIRONMENT
Android (APK / AAB) & iOS

TABLE OF CONTENTS

01. Executive Summary & Architecture Pg 2
02. Authentication & User Session Flow Pg 2
03. Dashboard & Operation Hub Pg 3
04. Quick Bill & Invoice Creation Flow Pg 3
05. Inventory Math & Combined Stock Algorithm Pg 4
06. Duplicate Line Item Prevention System Pg 4
07. Invoice Editing & Atomic Stock Deltas Pg 5
08. Product Inventory & Stock Movements Pg 5
09. Customer Directory & Payment Ledger Pg 6
10. Sales Analytics & Reporting Engine Pg 6
11. Printing Hardware & PDF Generation Pg 7
12. Backend API & Database Schema Pg 7

1. Executive Summary & System Architecture

D R Agencies Billing App is an enterprise-grade point-of-sale, GST billing, and inventory management mobile application developed for wholesale distributors and retail agencies. It eliminates manual billing errors, automates stock deduction, tracks customer credit ledgers, and connects wirelessly with portable Bluetooth thermal receipt printers.

Frontend Architecture

- **Expo Router v5**: File-based routing with deep-linking support.
- **Redux Toolkit (RTK)**: Centralized store management with asynchronous thunks.
- **AsyncStorage**: Local persistence for offline state and token caching.
- **React Native Reanimated**: Smooth 60fps animations and micro-interactions.

Backend Architecture

- **Express.js REST API**: Lightweight microservices controller layer.
- **Supabase PostgreSQL**: Scalable database with Row-Level Security (RLS).
- **Postgres RPC**: Atomic server-side stored procedures for inventory operations.
- **JWT Authentication**: Stateless bearer token verification.

2. Authentication & User Session Flow

The authentication system ensures secure multi-tenant isolation. All inventory items, customer lists, and invoice records are bound to the authenticated user's unique `user_id`.

1. **Login Screen (`app/login.tsx`)**
   Supports login via registered email or 10-digit mobile number with password verification. Form validation checks email formatting and password strength in real time.
2. **JWT Token Exchange & Persistence**
   Upon successful authentication, the backend issues a signed JWT token. The app saves the token into device `AsyncStorage` and initializes the global Redux auth slice.
3. **Auto-Hydration & Persistent Auth Guard**
   When the app is relaunched, `initializeAuth()` validates the cached token and immediately redirects authenticated users to the main Dashboard, bypassing login.

Endpoint	Method	Description	Payload / Auth
`/auth/login`	POST	Authenticates user and returns JWT + Store Profile	`{ email, password }`
`/auth/register`	POST	Registers new merchant business account	`{ email, password, name, phone }`
`/auth/profile`	GET	Fetches business GSTIN, address & banking details	`Bearer <token>`

3. Dashboard & Operations Hub

The Dashboard (`app/(tabs)/index.tsx`) serves as the daily operational center, rendering real-time business performance indicators, quick navigation shortcuts, and the live stream of recent invoices.

Today's Sales Revenue

₹ Live Calculated

Sum total of all invoices issued on current calendar date.

Invoices Billed

Live Count

Total count of bills generated today with status tags.

Inventory Alerts

Dynamic

Flags products with low stock (<10 kg) or 0 kg remaining.

Key Operations on Invoices in Dashboard

- **Invoice Status Badges**: Displays green `PAID` or amber `PENDING` indicator.
- 🖍️ **Edit Invoice**: Opens the invoice in `create-bill.tsx` in edit mode with pre-populated customer, item quantities, and stock allocation.
- 🖨️ **Thermal Print**: Directly prints an ESC/POS receipt to the connected Bluetooth printer without leaving the dashboard.
- 📄 **Download / View PDF**: Generates an A4 Tax Invoice PDF with GST breakup.
- 🗑️ **Delete with Stock Rollback**: Removes the invoice and automatically increments inventory stock back into the database.

4. Quick Bill & Invoice Creation Flow

The Quick Bill screen (`app/(tabs)/create-bill.tsx`) is engineered for high-speed counter checkout.

1. Customer Information & Auto-Fill

Typing in the customer name field opens an autocomplete dropdown from the Customer Directory. Selecting a customer auto-populates address, phone, GSTIN, and State. If a new customer is typed, a **"+ Save to Directory"** button appears to persist their details with a single tap.

2. Product Entry & Live Stock Badge

Searching a product dynamically displays its current store stock. When selected, the unit price is filled automatically. The quantity input supports decimal numbers (e.g. `2.5 kg`) for weight-based sales.

3. Dynamic GST & Payment Configurations

The GST toggle automatically computes CGST (50%) and SGST (50%) according to each product's configured tax rate (0%, 5%, 12%, 18%, 28%). Grand totals are rounded off cleanly. Payment status can be toggled to **Paid** with method selection (**Cash, GPay, PhonePe, Paytm**).

5. Combined Stock Allocation Algorithm

In a dynamic retail environment, editing an existing invoice requires a specialized mathematical formula. The system must account for stock that was already allocated to the current bill plus remaining stock in the physical store.

IN THIS BILL		STORE STOCK		MAX ALLOWED
1.0 kg	+	**4.0 kg**	=	**5.0 kg**

Mathematical Formulation

```
TotalStockPool      = DB_Stock_Qty + Original_Billed_Qty
RemainingStoreStock = MAX(0, TotalStockPool - Current_Active_Bill_Qty)
MaxAllowedForItem   = TotalStockPool - Other_Items_In_Bill_Qty
```

Concrete Scenario Walkthrough

1. **Initial State**: Product A has **5 kg** in store inventory.
2. **First Bill Created**: Customer purchases **1 kg**. Backend deducts 1 kg; DB stock becomes **4 kg**.
3. **Invoice Edit Session Opened**: The bill loads with 1 kg. Total available pool is `4 kg + 1 kg = 5 kg`.
4. **User Increments to 3 kg**: Stepper allows increment up to 5 kg. `3 kg ⩽ 5 kg` is valid.
5. **Save & Sync**: Backend restores 1 kg (DB=5 kg), validates 3 kg <= 5 kg, deducts 3 kg. Remaining DB stock becomes **2 kg**!

6. Duplicate Line Item Prevention System

Allowing duplicate rows for the same product in a single bill creates quantity calculation mismatches and invoice ambiguity. A multi-layered protection mechanism is active in `create-bill.tsx`:

1. Visual Dropdown Highlighting

Products already present in the active bill appear in the search dropdown with a distinctive gold highlight and a badge: `✓ Added in invoice (X kg) • Tap to edit below`.

2. Interception & Guidance Modal

If the user attempts to select or click "+ Add Item" for an already added product, the system intercepts the action and displays an informative alert: *"Product is already in this invoice. Please edit its quantity from the Items List below."*

7. Invoice Editing & Atomic Stock Deltas

The invoice update workflow utilizes database-level transactions to ensure stock counts are never corrupted or over-deducted during multi-user edits.

1. **Restore Previous Stock (Reversion)**
   The backend controller fetches the previous line items of the invoice and calls `decrement_product_stock(p_id, -old_qty)` to restore inventory temporarily.
2. **Aggregate Validation**
   The system sums total requested quantities per product ID and validates against the newly restored store stock. If requested quantity exceeds stock, the operation rolls back and returns a 400 error.
3. **Deduct New Stock & Replace Line Items**
   The backend deducts the new quantities atomically, replaces the rows in `bill_items`, updates bill totals, and records stock movement logs with reference `UPDATE-BILL-<inv_no>`.

8. Product Inventory & Stock Movements

The Products module (`app/(tabs)/products.tsx`) manages the central inventory catalog, stock-in entries, and audit logs.

Field	Type	Description
`name`	Text	Product title (e.g., "Hot & Spicy Masala 100g")
`hsn`	String	Harmonized System of Nomenclature code for tax classification
`price`	Decimal	Default selling unit price (₹ per kg / packet)
`gstRate`	Integer	Applicable GST percentage (0%, 5%, 12%, 18%, 28%)
`stock_qty`	Decimal	Live available stock on hand in kilograms / units

Stock In / Stock Out History Modal

Tapping any product opens the Stock Ledger Modal where store managers can add incoming stock batches with purchase reference notes. Every adjustment creates a timestamped row in `product_stock_logs` marked as **IN** (purchase/restock) or **OUT** (invoiced sale).

9. Customer Directory & Payment Ledger

The Customer Management system (`app/customers.tsx` & `app/(tabs)/payments.tsx`) tracks client directories, contact credentials, and credit ledgers.

Customer Directory

- **Quick Search**: Filter by customer name or phone number.
- **Duplicate Protection**: Blocks duplicate contact numbers.
- **GSTIN & State Tracking**: Supports B2B tax compliance.
- **Direct Dial**: One-tap calling to customer phone.

Customer Ledger & Balances

- **Total Billed**: Cumulative sum of all invoices for the customer.
- **Total Received**: Sum of payments received from the customer.
- **Outstanding Balance**: `Total Billed - Total Received`.
- **Payment Recording**: Record Cash, GPay, PhonePe, Paytm payments.

Customer Statement PDF Export

The Ledger screen includes a **"Download PDF Statement"** feature that formats all invoices and payments for a specific customer into a formal account statement with starting balance, transaction history, and outstanding balance summary.

10. Sales Analytics & Reporting Engine

The Reports screen (`app/(tabs)/reports.tsx`) provides executive sales insights and tax compliance summaries over customizable timeframes.

Filter Mode	Date Calculation	Primary Use Case
Today	Current calendar day (00:00 - 23:59)	End-of-day register balancing and cash reconciliation.
Weekly	Trailing 7 days	Weekly sales trends and high-demand product tracking.
Monthly	1st day of month to current date	Monthly revenue monitoring and tax filing prep.
Custom Range	Selected start and end date picker	Quarterly audits and financial year reporting.

11. Printing Hardware & PDF Generation

The application features a hybrid printing pipeline supporting both wireless portable Bluetooth thermal printers and standard A4 office printers.

Bluetooth ESC/POS Thermal Printing

- **Supported Sizes**: 58mm (2-inch) and 80mm (3-inch) rolls.
- **Native ESC/POS Commands**: Sends raw byte streams for bold headers, table alignment, cut commands, and barcodes.
- **Auto-Connect**: Remembers paired Bluetooth MAC addresses for instant printing.

A4 Tax Invoice PDF Engine

- **Expo Print & Sharing**: Compiles high-resolution HTML templates into PDF documents.
- **Branded Header**: Embeds store emblem, contact numbers, and GSTIN.
- **Rupees in Words**: Automatic conversion of numeric totals into Indian Rupees word format (Lakhs & Crores).

12. Backend API Reference & Supabase Database

The backend is powered by Node.js, Express, and Supabase PostgreSQL.

Route	Method	Function
`/bills/create`	POST	Validates inventory, inserts bill + items, decrements stock atomically.
`/bills/recent`	GET	Fetches the latest invoices with nested line items.
`/bills/:id`	GET	Retrieves single invoice detail including customer & item rows.
`/bills/:id`	PUT	Updates bill, restores old stock, validates & deducts new stock delta.
`/bills/:id`	DELETE	Deletes bill and automatically restores product stock quantities.
`/products/stock/add`	POST	Adds restock quantity and logs entry to `product_stock_logs`.
`/customers/payment`	POST	Records customer payment and updates customer `total_received`.

Postgres Atomic Stock RPC Function (`schema.sql`)

```
CREATE OR REPLACE FUNCTION decrement_product_stock(p_id BIGINT, p_qty NUMERIC, p_user_id UUID)
RETURNS BOOLEAN AS $$
DECLARE v_current_stock NUMERIC;
BEGIN
  SELECT stock_qty INTO v_current_stock FROM products WHERE id = p_id AND user_id = p_user_id FOR UPDA
  IF v_current_stock IS NULL OR (p_qty > 0 AND v_current_stock < p_qty) THEN
    RETURN FALSE;
  END IF;
  UPDATE products SET stock_qty = stock_qty - p_qty, updated_at = NOW() WHERE id = p_id AND user_id =
  RETURN TRUE;
END;
```

```
$$ LANGUAGE plpgsql SECURITY DEFINER;
```